WEEK 5 PROGRESS REPORT

1. Introduction

During Week 5, work on CampusConnect focused on implementing and testing a bounded agent workflow for handling student-support requests. The aim was to move beyond a system that only responds to questions and develop an agent that can select appropriate tools, perform tasks in stages, interpret tool results, and stop when a task is complete or when further action requires user approval.

The implementation uses LangGraph to organise the agent workflow and the Gemini model, gemini-3.8-flash, to support decision-making. Testing focused on support-request drafting, checking the status of existing requests, and ensuring that the agent respects approval requirements and handles unsuccessful tool operations appropriately.

2. Tasks Completed

2.1. Implementation of the Bounded Agent Workflow

The CampusConnect agent was organised into four LangGraph nodes: sense, decide, act, and stop. These nodes support the process of receiving a user's request, determining the appropriate next action, executing the selected tool operation, and ending the workflow when the task is complete or cannot proceed.

A maximum execution limit of five steps (MAX_STEPS = 5) was configured to restrict the number of iterations the agent can perform during a single workflow. This limit helps prevent uncontrolled execution and ensures that the agent operates within a defined boundary.

The workflow was tested through live executions using the configured Gemini model. The captured traces show the agent making tool-selection decisions, executing selected tools, interpreting their results, and producing final responses.

Outcome: The agent completed multiple stages of support-request workflows and recorded its internal execution events. The captured evidence includes model decisions, tool calls, tool results, and stopping reasons.

2.2. Description of the Agent Workflow

The CampusConnect agent follows a structured, multi-step workflow implemented using LangGraph. The workflow is organised into four nodes: sense, decide, act, and stop. Each node has a specific responsibility in processing a student-support request.

The workflow begins when a student submits a request through the application. The agent receives the request and gathers relevant context, including information retrieved from the CampusConnect knowledge base. The sense node supports this initial stage by preparing the information required for the agent to determine an appropriate response or action.

The request and available context are then passed to the decide node. At this stage, the Gemini model (gemini-3.8-flash) determines the next appropriate action. Depending on the request, the

agent may respond using the available evidence or select one of its available tools. For example, a student asking about an existing support request may require the get_support_request_status tool, while a student experiencing a registration problem may require create_support_request_draft.

When a tool is selected, the workflow proceeds to the act node, where the selected operation is executed. The result is returned to the agent and used to determine what should happen next. If the task requires another action, the agent can return to the decision stage and select the next appropriate step. This allows it to perform a sequence of related actions rather than relying on a single response.

Certain operations have additional restrictions, In particular, submitting a support request requires explicit user approval. If approval is denied, the submission is stopped and the agent informs the user that the request has not been submitted. Similarly, when a tool returns an error, such as NOT_FOUND for an unrecognised support-request reference, the agent can interpret the result and provide an appropriate explanation rather than treating the operation as successful.

The workflow ends at the stop node when the task is complete, an operation cannot proceed, approval is denied, or the configured execution limit is reached. The agent has a maximum execution limit of five steps (MAX_STEPS = 5), which places a boundary on the number of iterations it can perform. The workflow also records execution events, allowing decisions, tool calls, results, and stopping behaviour to be inspected through the captured traces.

Outcome: The workflow provides a controlled sequence for processing requests, selecting tools, interpreting results, and stopping safely. The live execution traces provide examples of this process through draft creation, approval denial, and handling of a missing support-request reference.

2.3. Integration of Student-Support Tools

The agent was connected to tools that allow it to carry out specific student-support operations. These include:

- lookup_lecture_venue for looking up lecture venue information.
- get_support_request_status for checking the status of an existing support request.
- create_support_request_draft for preparing a support request without immediately submitting it.
- submit_support_request for submitting a request, subject to explicit user approval.

These tools give the agent defined actions it can select according to the user's request. Creating a draft and sending a request to support staff are treated as separate operations because submission has consequences beyond preparing the request.

Outcome: Live tests confirmed that the agent could select and execute the support-request status and draft-creation tools. The submission test also confirmed that the agent respected the approval requirement when the user declined permission.

2.4. Testing Approval-Controlled Submission

A live test was conducted using the existing support request CC-2026-00002. The user asked the agent to check the request and follow the normal submission process.

The agent first selected get_support_request_status. The tool returned the request details and indicated that the request was still in DRAFT status. The agent then selected submit_support_request, which triggered the approval step.

When the user denied approval, the operation returned the error code APPROVAL_DENIED. The agent stopped the workflow and informed the user that the request had not been submitted.

Outcome: The test confirmed that the agent could check an existing request, proceed to the submission stage, and stop safely when permission was denied. The captured trace provides evidence that the submission operation did not proceed without approval.

2.5. Live Execution Trace Capture

A separate trace-capture script, app/capture_live_trace.py, was used to capture internal events from actual Gemini agent executions without modifying the original CLI application, app/campusconnect.py.

Three distinct execution traces were retained in evidence/week5/execution_trace/.

Trace file	Test scenario and observed outcome
live_session.json	The agent checked support request CC-2026-00002 and attempted to proceed with submission. The user denied approval, and the agent stopped without submitting the request. The trace contains 11 events.
draft_creation.json	The agent created draft request CC-2026-00003 for a course-registration problem involving missing courses. The request remained in DRAFT status and was not submitted. The trace contains 8 events.
failure_recovery.json	The agent checked the deliberately invalid reference CC-9999-99999. The status tool returned NOT_FOUND, after which the agent explained that the request could not be found and provided next steps. The trace contains 8 events.

The trace files preserve the recorded execution events rather than relying solely on the agent's final answers. They include the original user request, retrieved evidence, model decisions, tool calls, tool results, stopping behaviour, and final response.

Outcome: Three separate traces were captured from live Gemini executions. Together, they provide evidence of draft creation, approval-controlled stopping, and handling of a failed status lookup. The missing-reference test demonstrated failure handling; it did not recover the missing request itself.

3. Conclusion

During Week 5, CampusConnect progressed from a student-support assistant towards a bounded agent capable of selecting tools and carrying out multi-step support-request workflows. Live testing showed that the agent could create a support-request draft, check an existing request, respect a denied submission approval, and handle a missing-reference error without creating or submitting an unintended request.